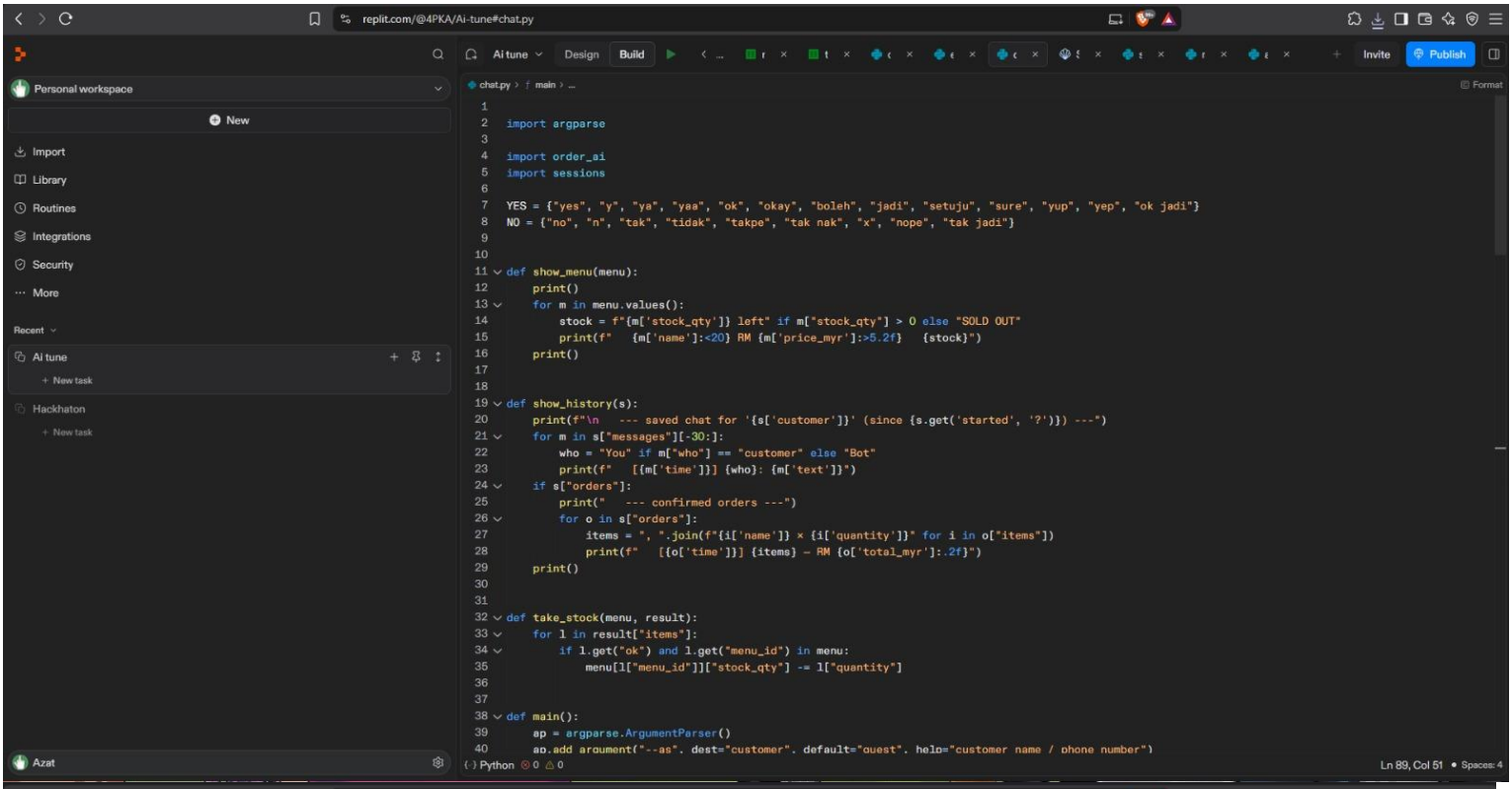


replit.com/@4PKA/Ai-tune#chat.py

Ai tune | Design | Build | Invite | Publish

Personal workspace

New

Import
Library
Routines
Integrations
Security
More

Recent

Ai tune
+ New task

Hackhaton
+ New task

Azat

chat.py > main > ...

Format

```python
import argparse

import order_ai
import sessions

YES = {"yes", "y", "ya", "yaa", "ok", "okay", "boleh", "jadi", "setuju", "sure", "yup", "yep", "ok jadi"}
NO = {"no", "n", "tak", "tidak", "takpe", "tak nak", "x", "nope", "tak jadi"}


def show_menu(menu):
    print()
    for m in menu.values():
        stock = f"{m['stock_qty']} left" if m["stock_qty"] > 0 else "SOLD OUT"
        print(f"  {m['name']:<20} RM {m['price_myr']:>5.2f}  {stock}")
    print()

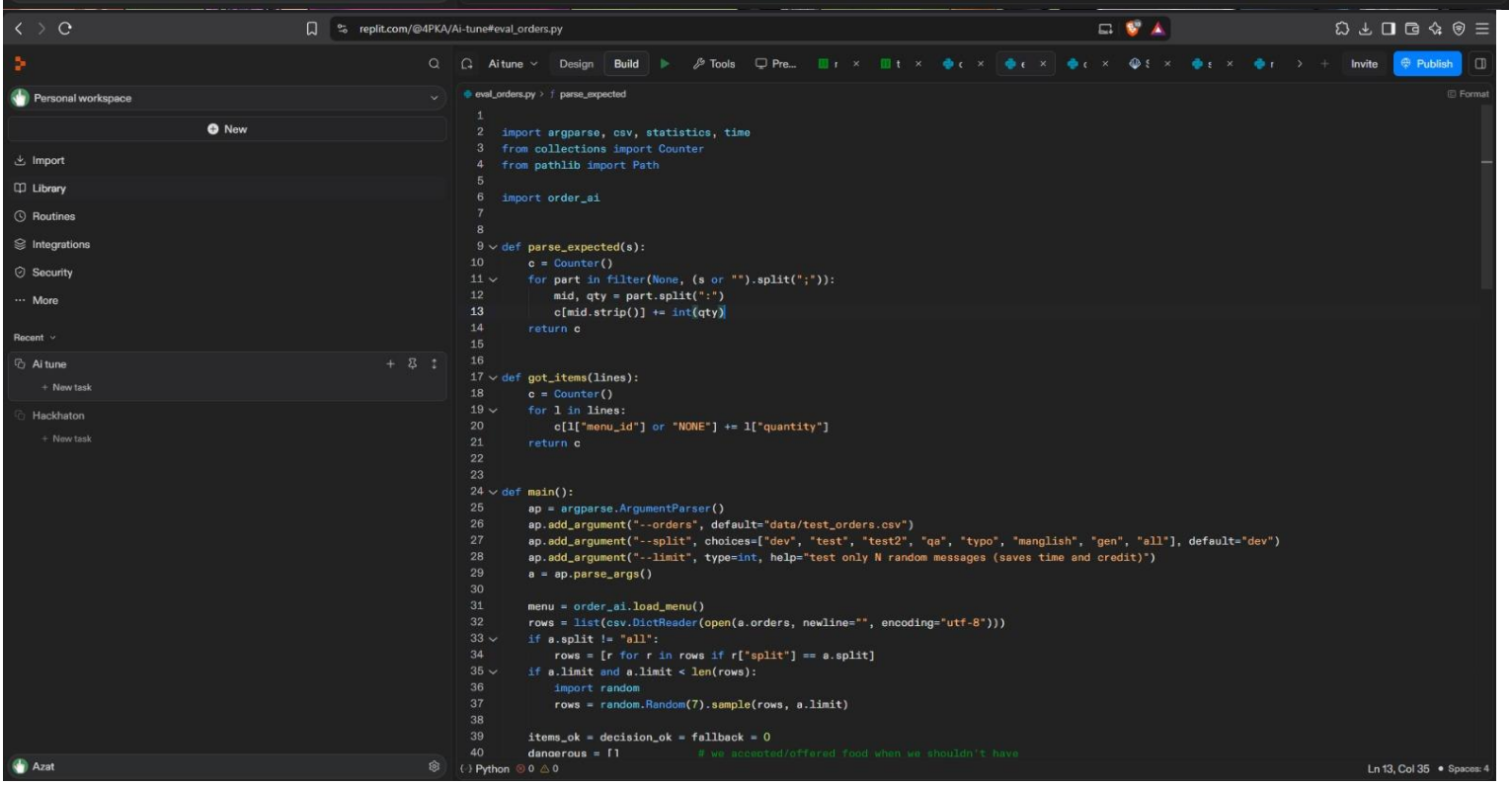

def show_history(s):
    print(f"\n  --- saved chat for '{s['customer']}' (since {s.get('started', '?')}) ---")
    for m in s["messages"][-30:]:
        who = "You" if m["who"] == "customer" else "Bot"
        print(f"  [{m['time']}] {who}: {m['text']}")
    if s["orders"]:
        print("  --- confirmed orders ---")
        for o in s["orders"]:
            items = ", ".join(f"{i['name']} x {i['quantity']}" for i in o["items"])
            print(f"  [{o['time']}] {items} - RM {o['total_myr']:.2f}")
    print()


def take_stock(menu, result):
    for l in result["items"]:
        if l.get("ok") and l.get("menu_id") in menu:
            menu[l["menu_id"]]["stock_qty"] -= l["quantity"]


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--as", dest="customer", default="guest", help="customer name / phone number")
```

Python 0 0

Ln 89, Col 51 • Spaces: 4



replit.com/@4PKA/Ai-tune#eval_orders.py

Ai tune | Design | Build | Tools | Pre... | Invite | Publish

Personal workspace

New

Import
Library
Routines
Integrations
Security
More

Recent

Ai tune
+ New task

Hackhaton
+ New task

Azat

eval_orders.py > parse_expected

Format

```python
import argparse, csv, statistics, time
from collections import Counter
from pathlib import Path

import order_ai


def parse_expected(s):
    c = Counter()
    for part in filter(None, (s or "").split(";")):
        mid, qty = part.split(":")
        c[mid.strip()] += int(qty)
    return c


def got_items(lines):
    c = Counter()
    for l in lines:
        c[l["menu_id"] or "NONE"] += l["quantity"]
    return c

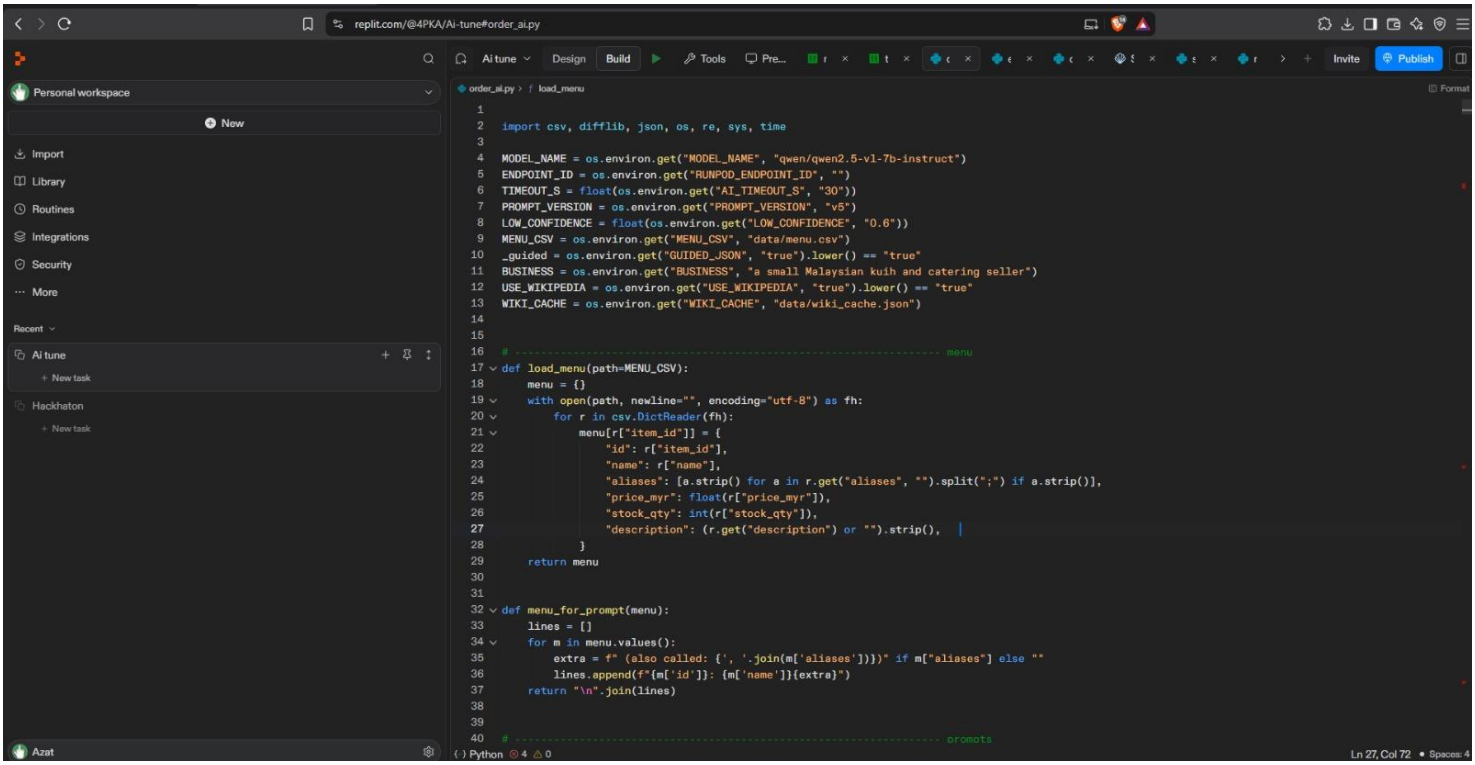

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--orders", default="data/test_orders.csv")
    ap.add_argument("--split", choices=["dev", "test", "test2", "qa", "typo", "manglish", "gen", "all"], default="dev")
    ap.add_argument("--limit", type=int, help="test only N random messages (saves time and credit)")
    a = ap.parse_args()

    menu = order_ai.load_menu()
    rows = list(csv.DictReader(open(a.orders, newline="", encoding="utf-8")))
    if a.split != "all":
        rows = [r for r in rows if r["split"] == a.split]
    if a.limit and a.limit < len(rows):
        import random
        rows = random.Random(7).sample(rows, a.limit)

    items_ok = decision_ok = fallback = 0
    dangerous = []          # we accepted/offered food when we shouldn't have
```

Python 0 0

Ln 13, Col 35 • Spaces: 4



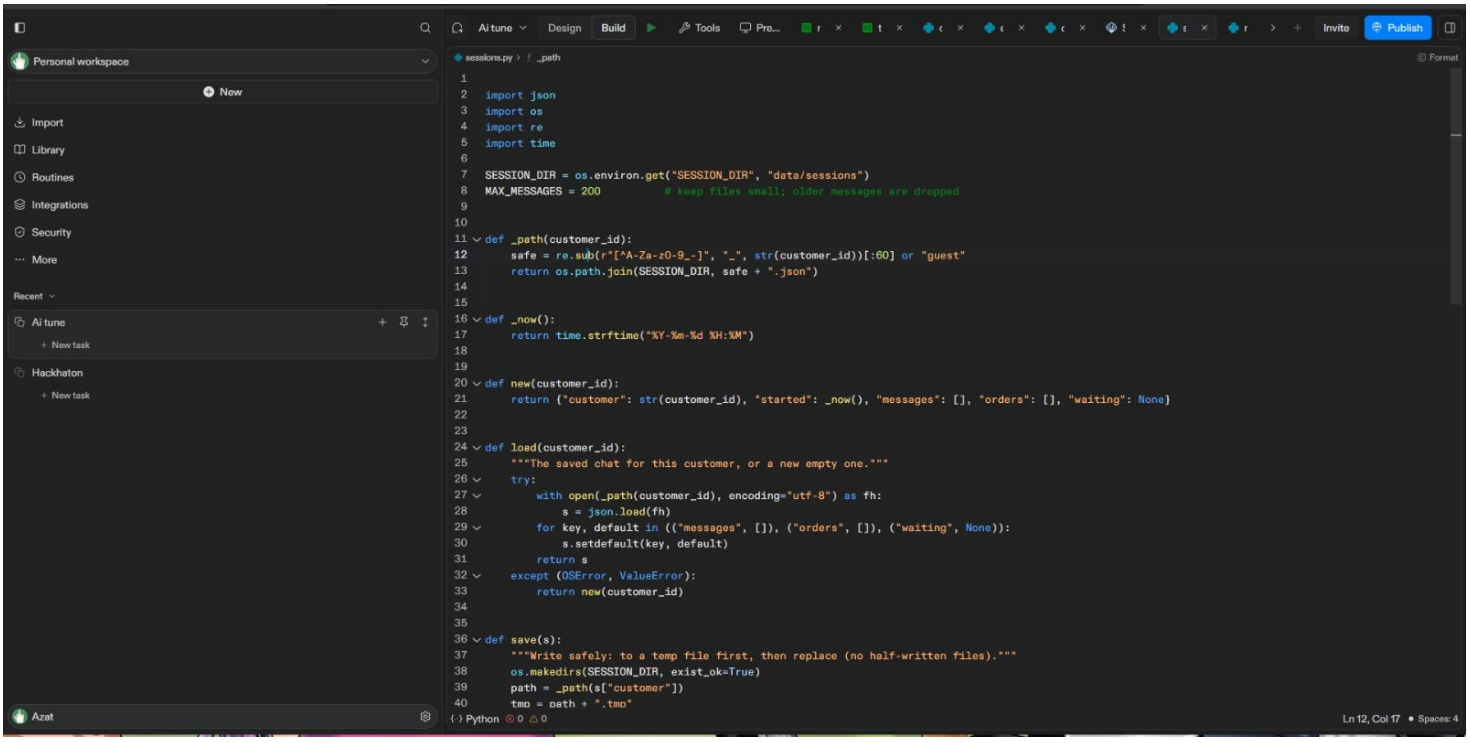
```
import csv, difflib, json, os, re, sys, time

MODEL_NAME = os.environ.get("MODEL_NAME", "qwen/qwen2.5-vl-7b-instruct")
ENDPOINT_ID = os.environ.get("RUNPOD_ENDPOINT_ID", "")
TIMEOUT_S = float(os.environ.get("AI_TIMEOUT_S", "30"))
PROMPT_VERSION = os.environ.get("PROMPT_VERSION", "v5")
LOW_CONFIDENCE = float(os.environ.get("LOW_CONFIDENCE", "0.6"))
MENU_CSV = os.environ.get("MENU_CSV", "data/menu.csv")
_guided = os.environ.get("GUIDED_JSON", "true").lower() == "true"
BUSINESS = os.environ.get("BUSINESS", "a small Malaysian kuih and catering seller")
USE_WIKIPEDIA = os.environ.get("USE_WIKIPEDIA", "true").lower() == "true"
WIKI_CACHE = os.environ.get("WIKI_CACHE", "data/wiki_cache.json")


# ---------------------------------------------------------------- menu
def load_menu(path=MENU_CSV):
    menu = {}
    with open(path, newline="", encoding="utf-8") as fh:
        for r in csv.DictReader(fh):
            menu[r["item_id"]] = {
                "id": r["item_id"],
                "name": r["name"],
                "aliases": [a.strip() for a in r.get("aliases", "").split(";") if a.strip()],
                "price_myr": float(r["price_myr"]),
                "stock_qty": int(r["stock_qty"]),
                "description": (r.get("description") or "").strip(),
            }
    return menu


def menu_for_prompt(menu):
    lines = []
    for m in menu.values():
        extra = f" (also called: {', '.join(m['aliases'])})" if m["aliases"] else ""
        lines.append(f"{m['id']}: {m['name']}{extra}")
    return "\n".join(lines)


# ---------------------------------------------------------------- prompts
```



```
import json
import os
import re
import time

SESSION_DIR = os.environ.get("SESSION_DIR", "data/sessions")
MAX_MESSAGES = 200          # keep files small; older messages are dropped


def _path(customer_id):
    safe = re.sub(r"[^A-Za-z0-9_-]", "_", str(customer_id))[:60] or "guest"
    return os.path.join(SESSION_DIR, safe + ".json")


def _now():
    return time.strftime("%Y-%m-%d %H:%M")


def new(customer_id):
    return {"customer": str(customer_id), "started": _now(), "messages": [], "orders": [], "waiting": None}


def load(customer_id):
    """The saved chat for this customer, or a new empty one."""
    try:
        with open(_path(customer_id), encoding="utf-8") as fh:
            s = json.load(fh)
        for key, default in (("messages", []), ("orders", []), ("waiting", None)):
            s.setdefault(key, default)
        return s
    except (OSError, ValueError):
        return new(customer_id)


def save(s):
    """Write safely: to a temp file first, then replace (no half-written files)."""
    os.makedirs(SESSION_DIR, exist_ok=True)
    path = _path(s["customer"])
    tmp = path + ".tmp"
```

Personal workspace

New

Import

Library

Routines

Integrations

Security

More

Recent

Ai tune

+ New task

Hackhaton

+ New task

Azat

Ai tune | Design | Build | Invite | Publish

make_training_data.py

Format

```python

import csv
import json
import os
import random
from collections import Counter

import order_ai

random.seed(42)
OUT = "data/train"

MAIN_MENU = order_ai.load_menu()

ALT_MENUS = {
    "burger": ("a small burger stall in Kuala Lumpur", {
        "B01": {"id": "B01", "name": "Chicken Burger", "aliases": ["burger ayam"], "price_myr": 6.5, "stock_qty": 20, "description": ""},
        "B02": {"id": "B02", "name": "Beef Burger", "aliases": ["burger daging"], "price_myr": 7.5, "stock_qty": 12, "description": ""},
        "B03": {"id": "B03", "name": "Fries", "aliases": ["french fries", "kentang goreng"], "price_myr": 4.0, "stock_qty": 30, "description": ""},
        "B04": {"id": "B04", "name": "Hot Dog", "aliases": ["hotdog"], "price_myr": 5.0, "stock_qty": 0, "description": ""},
        "B05": {"id": "B05", "name": "Iced Lemon Tea", "aliases": ["ice lemon tea", "teh o ais limau"], "price_myr": 3.0, "stock_qty": 40,
"description": ""},
        "B06": {"id": "B06", "name": "Milo Ais", "aliases": ["iced milo", "milo"], "price_myr": 3.5, "stock_qty": 25, "description": ""},
    }),
    "kopitiam": ("a kopitiam (coffee shop) in Penang", {
        "D01": {"id": "D01", "name": "Kopi O", "aliases": ["kopi-o", "black coffee"], "price_myr": 2.2, "stock_qty": 50, "description": ""},
        "D02": {"id": "D02", "name": "Teh Ais", "aliases": ["iced tea", "teh peng"], "price_myr": 2.8, "stock_qty": 50, "description": ""},
        "D03": {"id": "D03", "name": "Roti Bakar", "aliases": ["toast", "roti kaya"], "price_myr": 3.0, "stock_qty": 18, "description": ""},
        "D04": {"id": "D04", "name": "Half Boiled Egg", "aliases": ["telur separuh masak", "soft boiled egg"], "price_myr": 2.5, "stock_qty": 24,
"description": ""},
        "D05": {"id": "D05", "name": "Nasi Lemak Ayam", "aliases": ["nasi lemak chicken"], "price_myr": 9.0, "stock_qty": 0, "description": ""},
        "D06": {"id": "D06", "name": "Cendol", "aliases": [], "price_myr": 4.5, "stock_qty": 15, "description": ""},
    }),
}

OFF_MENU = ["burger", "pizza", "sushi", "roti canai", "satay", "cendol", "nasi goreng", "laksa", "roti john",
            "char kway teow", "kopi o", "ais kacang", "mee rebus", "rojak", "chicken rice", "fries", "milo",
            "karipap", "teh tarik", "dim sum", "nasi kandar", "apam balik"]

UNITS = {"default": ["pcs", "biji"], "rice": ["bungkus", "packs"], "drink": ["cawan", "cups"], "slice": ["keping"]}
```

Python | Ln 1, Col 1 | Spaces: 4

Personal workspace

New

Import

Library

Routines

Integrations

Security

More

Recent

Ai tune

+ New task

Hackhaton

+ New task

Azat

Ai tune | Design | Build | Invite | Publish

sessions.py > _path

Format

```python

import json
import os
import re
import time

SESSION_DIR = os.environ.get("SESSION_DIR", "data/sessions")
MAX_MESSAGES = 200          # keep files small; older messages are dropped


def _path(customer_id):
    safe = re.sub(r"[^A-Za-z0-9_-]", "_", str(customer_id))[:60] or "guest"
    return os.path.join(SESSION_DIR, safe + ".json")


def _now():
    return time.strftime("%Y-%m-%d %H:%M")


def new(customer_id):
    return {"customer": str(customer_id), "started": _now(), "messages": [], "orders": [], "waiting": None}


def load(customer_id):
    """The saved chat for this customer, or a new empty one."""
    try:
        with open(_path(customer_id), encoding="utf-8") as fh:
            s = json.load(fh)
        for key, default in (("messages", []), ("orders", []), ("waiting", None)):
            s.setdefault(key, default)
        return s
    except (OSError, ValueError):
        return new(customer_id)


def save(s):
    """Write safely: to a temp file first, then replace (no half-written files)."""
    os.makedirs(SESSION_DIR, exist_ok=True)
    path = _path(s["customer"])
    tmp = path + ".tmp"
```

Python | Ln 12, Col 17 | Spaces: 4